

django

REST

framework

ENTORN DE TREBALL

Caldrà crear un nou projecte de **django 5.0**(el projecte de la documentació està fet amb Django 5.0) amb l'interpretador de Python 3.10. **NO UTILITZAR** el projecte de Django creat amb anterioritat.

S'aprendrà a crear una APIREST amb DRF (Django Rest Framework). La base inicial de Django és essencial per entendre les següents passes amb DRF.

NOMÉS EN CAS D'ERRORS i/o PROBLEMES!!!

- Seguir les passes a [IDE](#) si encara hi teniu problemes amb l'entorn.
- Seguir les passes del document de [DOCKER](#) si teniu problemes.

INSTAL·LACIÓ

Es segueixen les instruccions d'instal·lació de la documentació oficial de [DRE](#).

Requirements

REST framework requires the following:

- Django (4.2, 5.0)
- Python (3.8, 3.9, 3.10, 3.11, 3.12)

We **highly recommend** and only officially support the latest patch release of each Python and Django series.

The following packages are optional:

- **PyYAML**, **uritemplate** (5.1+, 3.0.0+) - Schema generation support.
- **Markdown** (3.3.0+) - Markdown support for the browsable API.
- **Pygments** (2.7.0+) - Add syntax highlighting to Markdown processing.
- **django-filter** (1.0.1+) - Filtering support.
- **django-guardian** (1.1.1+) - Object level permissions support.

Installation

Install using **pip**, including any optional packages you want...

```
pip install djangorestframework
pip install markdown          # Markdown support for the browsable API.
pip install django-filter     # Filtering support
```

...or clone the project from github.

```
git clone https://github.com/encode/django-rest-framework
```

Add **'rest_framework'** to your **INSTALLED_APPS** setting.

```
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

font: <https://www.django-rest-framework.org/#quickstart>

Llegir atentament les captures de la Installation ja que NO son 3 passes, son 2. Cal escollir entre les dues primeres i després fer la tercera. Es recomana fer la primera passa. La segona (clonar github) només si no s'aconsegueix de cap de les maneres que la primera passa funcioni.

També es recomana utilitzar **python3 -m** abans de **pip** de cada comanda. Si amb windows no us funciona, només usar **pip**.

Un cop instal·lat caldrà afegir a l'arxiu **settings.py** del projecte a la secció **INSTALLED_APPS** la llibreria instal·lada de **rest_framework**.

Caldrà també, a **settings.py**, modificar la secció de **DATABASE** per afegir la connexió adequada segons el docker, com ja s'ha vist amb l'activitat anterior.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest-framework'  
]
```

En el següent [enllaç](#) queda més clares les passes a seguir i com quedaria l'arbre del projecte de APIREST.

En comptes de posar-se a copiar codi i programar caldria entendre per a què serveix el [SERIALIZER](#). Us sonarà del **ModelForm** explicat a la teoria de **FORM**.

Ja heu de tenir una mica clar o us ha de sonar el concepte de **serialitzador**. Mireu a l'exemple que us posa que s'està utilitzant l'**HyperlinkedModelSerializer**. Tanmateix es pot utilitzar altres relacions (serializer / [ModelSerializer](#)) per entendre millor l'exemple.

DOCUMENTACIÓ

Per a fer una ApiRest amb DRF els inicis són els mateixos que en Django però sense l'ús de **templates** i afegint alguns nous conceptes com (**@api_views** i **serializers**). La resta de conceptes (que s'expliquen aquí) son aplicables a un projecte de back-end.

DJANGO MODELS: relacions entre models

Els **models** de Django són classes Python que defineixen l'estructura de les taules a la Base de dades. Les relacions entre models es gestionen mitjançant camps especials com **ForeignKey** i **ManyToManyField**.

- **ForeignKey**

Una relació **ForeignKey** estableix una relació **molts a un** entre dos models. Per exemple, una categoria pot tenir molts productes i un producte només pot pertànyer a una categoria.

Per tant, en un projecte de Django, una relació d'aquest tipus s'hauria de veure de la següent manera:

```
class Category(models.Model): 2 usages
    name = models.CharField(max_length=30)

class Product(models.Model):
    name = models.CharField(max_length=30)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    category= models.ForeignKey(Category, on_delete=models.CASCADE)
```

Category: representa una categoria de productes

Product: representa un producte. Té un camp **category** que estableix la relació amb el model **Category**.

on_delete=models.CASCADE: Defineix que, en cas d'eliminar-se una categoria, tots els productes associats a aquesta categoria també s'eliminen.

Aquesta relació ens permetrà obtenir informació des de categoria o des de producte. Es procedeix a posar uns exemples:

Obtenir tots els productes d'una categoria:

Python

```
category = Category.objects.get(id=1)
products = category.product_set.all()
```


Obtenir la categoria d'un producte:

Python

```
product = Product.objects.get(id=1)
category = product.category
```

- **ManyToManyField**

Una relació **ManyToManyFields** estableix una relació **molts a molts** entre dos models. Per exemple, un producte pot pertànyer a moltes categories.

```
class SuperProduct(models.Model):
    name = models.CharField(max_length=30)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    category = models.ManyToManyField(Category)
```

Imatge mostrant com es veu la migració:

```
class Migration(migrations.Migration):

    dependencies = [
        ('users', '0002_alter_user_role'),
    ]

    operations = [
        migrations.CreateModel(
            name='Category',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('name', models.CharField(max_length=30)),
            ],
        ),
        migrations.CreateModel(
            name='Product',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('name', models.CharField(max_length=30)),
                ('price', models.DecimalField(decimal_places=2, max_digits=10)),
                ('category', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='users.category')),
            ],
        ),
        migrations.CreateModel(
            name='SuperProduct',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('name', models.CharField(max_length=30)),
                ('price', models.DecimalField(decimal_places=2, max_digits=10)),
                ('category', models.ManyToManyField(to='users.category')),
            ],
        ),
    ]
```

Al crear una relació **Many To Many** la base de dades crea una taula intermitja per gestionar la relació **molts a molts**. Aquesta taula intermitja conté les claus primàries dels models relacionats (**product_id** i **category_id**).

```
> users_category
> users_product
> users_superproduct
> users_superproduct_category
```

Data Output			Messages	Notifications
≡+	📄	▼	📋	▼
	🗑️	🗄️	📥	📈
	id [PK] bigint	superproduct_id bigint	category_id bigint	

SERIALIZERS

La classe **serializer** serveix per transformar / modificar / serialitzar la resposta de la DB (format llista) en un format vàlid pel client (json).

El funcionament d'aquests **serializers** és molt similar al del **modelForm** de Django.

```
from rest_framework import serializers
from .models import User

class UserSerializer(serializers.Serializer):
    id = serializers.IntegerField(read_only=True)
    name = serializers.CharField(required=True, max_length=30)
    surname = serializers.CharField(required=False, max_length=30)
    age = serializers.IntegerField()
    rol = serializers.CharField(max_length=30, choices=User.Rol.choices)
```

Per paràmetre de la classe **UserSerializer** se li envia la funció **serializer** per a que pugui heretar tots els seus mètodes. En aquest serialitzador se li indica les dades a serialitzar. Es pot tenir diversos serialitzador per adaptar-se a la resposta que se li vulgui enviar al client.

Alternativa amb ús de **ModelSerializer**:

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'name', 'surname', 'age', 'rol']
        read_only_fields = ['id']
```

La lògica interna de la classe és la mateixa que la utilitzada al **modelForm**. Cal destacar que, independentment de la quantitat de camps que tingui el model de **Cataleg** la informació que serialitzarà serà la indicada a **fields**.

Més endavant, dintre del mètode de views, caldrà aprofitar aquest serialitzador per, amb la informació extreta de la **query list**, adaptar la resposta a un json amb els camps indicats a **fields**.

Més info: <https://www.django-rest-framework.org/tutorial/1-serialization/>

@api_views

Per a que **DJANGO REST FRAMEWORK** treballi com un APIREST, cal que se li defineixin uns endpoints (**urls.py**) i cal indicar-li al projecte quin mètode HTTP es farà servir (GET, POST, PUT, DELETE, etc.) amb **@api_view**.

@api_view és un decorador que s'encarrega d'indicar al projecte quin mètode HTTP ha de renderitzar per mostrar la resposta per l'explorador.

Més informació: https://www.django-rest-framework.org/api-guide/views/#api_view

Exemple on es renderitzarà a l'explorador el mètode GET:

```
@api_view(['GET']) 1 usage
def getAllUsers(request):
    if request.method == 'GET':
        users = User.objects.all()
        serializer = UserSerializer(users, many=True)
        return Response(serializer.data)
```

Serializer = UserSerializer(user, many = True) ⇒ es crea un objecte complex amb molta informació sobre la consulta i les dades de la base de dades

Serializer.data ⇒ Envia només les dades de l'objecte corresponent a la informació de la taula de la base de dades.

A **@api_view** es pot indicar més d'una acció. En el següent exemple s'utilitzarà **GET** i **POST**:

```
@api_view(['GET', 'POST'])
def users(request):
    if request.method == 'GET':
        users = User.objects.all()
        serializer = UserSerializer(users, many=True)
        return Response(serializer.data)

    if request.method == 'POST':
        serializer = UserSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)
```

Estandardització mètodes

En comptes de que cadascú s'inventi el nom d'un mètode, per tal de que quedi més clar (per a una transició a **ViewSet**) per a cada path s'hi haurà de crear un mètode amb aquell nom com a la llista següent, seguint la convenció de noms de **DRF**:

- GET "users/" ⇒ es crearà un mètode de nom **list**.
- GET "users/id/" ⇒ es crearà un mètode de nom **retrieve**.
- POST "user/" ⇒ es crearà un mètode de nom **create**.
- PUT "users/id/" ⇒ es crearà un mètode de nom **update**.
- PATCH "users/id/" ⇒ es crearà un mètode de nom **partial_update**.
- DELETE "users/id/" ⇒ es crearà un mètode de nom **destroy**.

Més informació sobre **serializer.errors**: [serializer.errors](#)

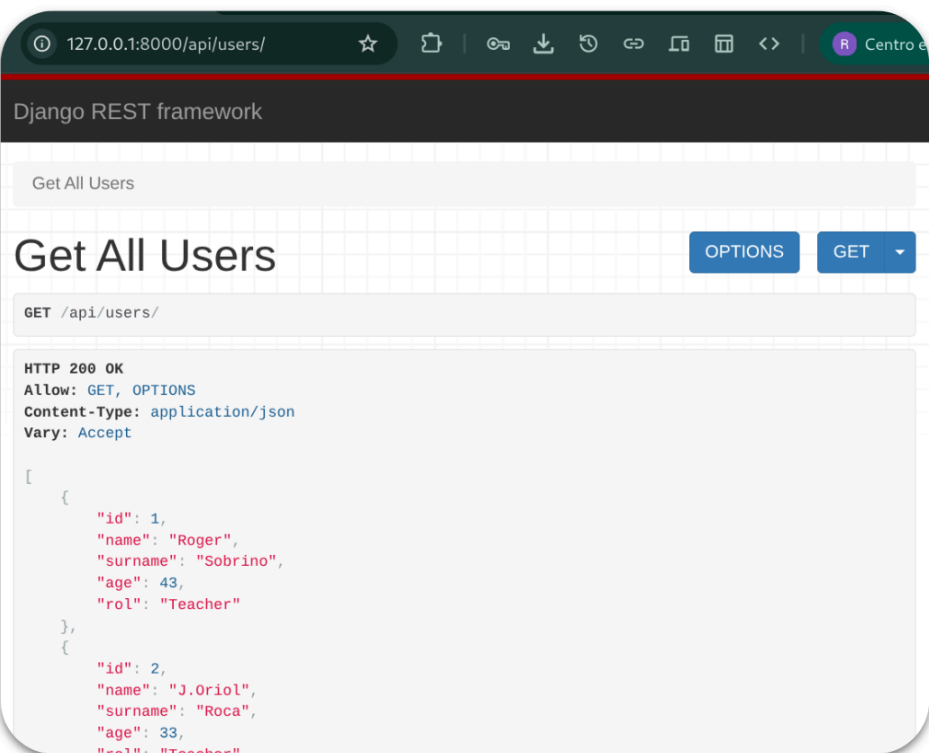
RECOMANACIONS

Existeixen les opcions d'ús de Routers amb **ViewSet** o **ModelViewSet**, maneres de programar més ràpides i senzilles però més complicades d'entendre. És **OBLIGATORI** utilitzar **@api_views** per a qualsevol activitat o projecte fet a **DSAW**. És més llarg, però es té més control de la resposta que se li vol enviar al client.

EXECUCIÓ

Amb la comanda **python3 manage.py runserver** i indicant a la url el path correcte, ha de sortir una imatge semblant a la següent:

Imatge corresponent al mètode **getAllUsers** amb l'acció **GET**.



Imatge corresponent al mètode **users** amb les accions **GET** i **POST**:

Users

OPTIONS

GET

GET /api/gp_users/

HTTP 200 OK

Allow: OPTIONS, GET, POST

Content-Type: application/json

Vary: Accept

```
[
  {
    "id": 1,
    "name": "Roger",
    "surname": "Sobrinó",
    "age": 43,
    "rol": "Teacher"
  },
  {
    "id": 2,
    "name": "J.Oriol",
    "surname": "Roca",
    "age": 33,
    "rol": "Teacher"
  },
  {
    "id": 3,
    "name": "Fermin",
    "surname": "Siviero",
    "age": 20,
    "rol": "Student"
  }
]
```

Media type:

application/json

Content:

POST

PERMISSIONS (POLICIES)

Cal crear l'arxiu **permissions.py** a la carpeta de l'aplicació. En aquest arxiu s'hi crearan els permisos (policies) que té cada rol en el nostre projecte:

Es vol donar els següents permisos:

- Student
 - Només pot llegir un registre d'una taula (**retrieve**) o la taula sencera (**list**)
- Teacher
 - Pot accedir a tot arreu
- Invitat
 - Només pot llegir

```
class RolePermission(permissions.BasePermission): 2 usages
    ''' OPCIO 1 MÉS EPSECÍFICA '''
    def has_permission(self, request, view):
        user = request.user

        # Si no està loguejat només pot usar GET
        if not user.is_authenticated:
            return request.method in permissions.SAFE_METHODS

        # Restriccions per Student
        if user.rol == "Student":
            if request.method == "GET":
                return True

        # El Rol de teacher ho pot fer tot
        if user.rol == "Teacher":
            return True

        return False
```

En el codi es pot veure com s'han repartit els permisos segons el rol.

permissions.SAFE_METHODS ⇒ indica el rol d'**invitat** només pot fer les accions **GET** (consultes a la base de dades).

La política de permisos permet mètodes de només lectura per a usuaris anònims mitjançant la validació de **SAFE_METHODS**. Mentre que un **GET** supera aquesta **validació**, un mètode d'escriptura com el **POST** és **interceptat** i denegat, obligant l'usuari a estar autenticat i posseir un rol autoritzat per prosseguir.

Afegir la Policie al mètode

Un cop creada la restricció (**Policie**) cal indicar aquesta policie a on es vol aplicar:

```
from rest_framework.decorators import api_view, permission_classes

@api_view(['GET']) 1 usage
@permission_classes([RolePermission])
def getAllUsers(request):
    if request.method == 'GET':
        users = User.objects.all()
        serializer = UserSerializer(users, many=True)
        return Response(serializer.data)
```